Lazizbek. MATH 548. HW # 5.

Solve the following equations using the fixed point iteration method. In each case the starting values are 2, 0.8, 0.2.

Pr. 1.

(1) $x = x^2 \quad \Rightarrow \quad x = g(x) \quad \Rightarrow$

(2) $g(x) = x^2$.

We know that for x_n

(3) $$x_{n+1} = x_n^2$$

fixed point iteration to be convergent

(4) $$|g'(x)| < 1$$

If we solve this (4), we find the convergent interval of (3):

(5) $|g'(x)| = |2x| < 1 \quad \Rightarrow \quad -1 < 2x < 1 \quad \Leftrightarrow$

(6) $x \in (-\frac{1}{2}, \frac{1}{2})$ = convergent interval for (3).

So if we start $x_0 \in (-0.5, 0.5)$ then (3) converges otherwise diverges, also $|g'(x_0)|$ can tell us how fast/slow (3) converges or diverges:

Let $x_0 = 2 \notin (-0.5, 0.5)$, so we can see

(7) $x_{n+1} = x_n^2$, $x_0 = 2$ diverges

(8) $|g'(x_0)| = |g'(2)| = |2x_0| = |2 \cdot 2| = 4 > 1 \Rightarrow$ (3) diverges.

(9) $\begin{cases} x_0 = 0.8 \notin (-0.5, 0.5), \text{ so} \\ x_{n+1} = x_n^2, \quad x_0 = 0.8 \text{ diverges as} \end{cases}$

(10) $|g'(x_0)| = |2x_0| = |2 \cdot 0.8| = 1.6 > 1 \Rightarrow$ (3) diverges.

yet,

(11) $\begin{cases} x_0 = 0.2 \in (-0.5, 0.5) \\ x_{n+1} = x_n^2, \quad x_0 = 0.2 \text{ converges as} \end{cases}$

(12) $|g'(x_0)| = |2x_0| = |2 \cdot 0.2| = 0.4 < 1 \Rightarrow$ (3) converges.

Rate of convergence/divergence is determined by

$4 > 1.6 > 1 \iff$

(13) $\begin{cases} |g''(2)| = 4 > 1 \Rightarrow (7) \text{ diverges faster than } (9), \\ |g'(0.8)| = 1.6 > 1 \Rightarrow (9) \text{ diverges slower than } (7), \\ |g'(0.2)| = 0.4 < 1 \Rightarrow (11) \text{ converges, nothing to compare.} \end{cases}$

You can see the number of steps to converge or diverge within the given tolerance in the program, attached at the end.

For the rest of the problems, let me not to be so detailed where I'll be calculating only $|g'(x_0)|$ to identify the rate of convergence/divergence to compare with the program:

Pr2.

$$x = \sqrt{x}$$

(14) $x_{n+1} = \sqrt{x_n} \Rightarrow$

(15) $g(x) = \sqrt{x} \Rightarrow$

(16) $g'(x) = \dfrac{1}{2\sqrt{x}} \Rightarrow$

(17) $|g'(x)| = \left|\dfrac{1}{2\sqrt{x}}\right| < 1 \Leftrightarrow -1 < \dfrac{1}{2\sqrt{x}} < 1 \Leftrightarrow \overbrace{-2 < \dfrac{1}{\sqrt{x}} < 2}^{x \neq 0,\, x \in R} \Rightarrow$

$\Leftrightarrow \dfrac{1}{\sqrt{x}} < 2 \Leftrightarrow \sqrt{x} > \dfrac{1}{2} \overset{(18)}{\Leftrightarrow} \boxed{x > \dfrac{1}{4}}$ convergence interval for (14).

(19) $\begin{cases} x_0 = 2 \in \left(\frac{1}{4}, \infty\right) \Rightarrow (14) \text{ converges "faster"} \\ |g'(x_0)| = \left|\dfrac{1}{2\sqrt{x_0}}\right| = \left|\dfrac{1}{2\sqrt{2}}\right| = 0.35 < 1 \end{cases}$

(20) $\begin{cases} x_0 = 0.8 \in (0.25, \infty) \Rightarrow (14) \text{ converges "slower"} \\ |g'(x_0)| = \left|\dfrac{1}{2\sqrt{0.8}}\right| = 0.56 < 1 \end{cases}$

(21) $\begin{cases} x_0 = 0.2 \notin (0.25, \infty) \Rightarrow (14) \text{ diverges.} \\ |g'(x_0)| = \left|\dfrac{1}{2\sqrt{0.2}}\right| = 1.11 > 1 \end{cases}$

Rate of convergence / divergence is as followse

$0.35 < 0.56 < 1 \Rightarrow$ (19) converges faster than (20).

Pr 3.

(22) $x = x + 2(x - x^2) \Rightarrow$

(23) $x_{n+1} = x_n + 2(x_n - x_n^2) \Rightarrow$

(24) $g(x) = x + 2(x - x^2) = x + 2x - 2x^2 = 3x - 2x^2$

(25) $g'(x) = 3 - 4x$

(26) $|g'(x)| = |3 - 4x| < 1 \iff -1 < 3 - 4x < 1 \Rightarrow$

(27) $\Rightarrow \boxed{\frac{1}{2} < x < 1}$ convergence interval for (23).

(28) $\begin{cases} x_0 = 2 \notin \left(\frac{1}{2}, 1\right) \Rightarrow (23) \text{ diverges "faster"} \\ |g'(x_0)| = |3 - 4x_0| = |3 - 8| = 5 > 1 \end{cases}$

(29) $\begin{cases} x_0 = 0.8 \in \left(\frac{1}{2}, 1\right) \Rightarrow (23) \text{ converges} \\ |g'(x_0)| = |3 - 4x_0| = |3 - 4 \cdot 0.8| = 0.2 < 1 \end{cases}$

(30) $\begin{cases} x_0 = 0.2 \notin \left(\frac{1}{2}, 1\right) \Rightarrow (23) \text{ diverges "slower"} \\ |g'(x_0)| = |3 - 4 \cdot 0.2| = 2.2 > 1 \end{cases}$

Rate of convergence/divergence

$5 > 2.2 > 1 \Rightarrow$ (28) diverges faster than (30).

Pr. 4.

(31) $x = x - \dfrac{x - x^2}{1 - 2x} \Rightarrow$

(32) $x_{n+1} = x_n - \dfrac{x_n - x_n^2}{1-2x_n} \Rightarrow$

(33) $g(x) = x - \dfrac{x - x^2}{1-2x}$

(34) $|g'(x)| = \left| 1 - \dfrac{(1-2x)(1-2x) - (x-x^2)\cdot(-2)}{(1-2x)^2} \right| < 1 \Leftrightarrow$

(35) $\left| \dfrac{(1-2x)^2 - (1-2x)^2 - 2(x-x^2)}{(1-2x)^2} \right| = \left| \dfrac{-2(x-x^2)}{(1-2x)^2} \right| < 1$

As (35) is difficult to solve directly, let me calculate & check $|g'(x_0)| < 1$ separately:

(36) $\begin{cases} x_0 = 2 \\ |g'(x_0)| = \left| \dfrac{-2(x_0 - x_0^2)}{(1-2x_0)^2} \right| = \left| \dfrac{2(2-4)}{(1-4)^2} \right| = \left| \dfrac{4}{9} \right| = \dfrac{4}{9} < 1 \end{cases}$

$\Rightarrow$ (32) converges at $x_0 = 2$ faster than $x_0 = 0.8$, $x_0 = 0.2$:

(37) $\begin{cases} x_0 = 0.8 \\ |g'(x_0)| = \left| \dfrac{-2(x_0 - x_0^2)}{(1-2x_0)^2} \right| = \left| \dfrac{-2(0.8 - 0.8^2)}{(1-2\cdot 0.8)^2} \right| = \left| \dfrac{2\cdot 0.16}{0.36} \right| = \dfrac{8}{9} < 1 \end{cases}$

$\Rightarrow$ (32) converges

(38) $\begin{cases} x_0 = 0.2 \\ |g'(x_0)| = |g'(0.2)| = \left| \dfrac{-2(0.2 - 0.2^2)}{(1-2\cdot 0.2)^2} \right| = \left| \dfrac{2\cdot 0.16}{0.36} \right| = \dfrac{8}{9} < 1 \end{cases}$

(32) converges

$\dfrac{4}{9} < \dfrac{8}{9} = \dfrac{8}{9} < 1 \Rightarrow x_0 = 2$ has a faster convergence.

math548_hw5_Lazizbek

February 21, 2024

1 Lazizbek Sadullaev, Math 548, HW #5

#Homework-5

Solve the following equations using the fixed point iteration method. In each case use the starting values as 2, 0.8 and 0.2.

1. x = x^2
2. x = x^(1/2)
3. x = x + 2*(x - x^2)
4. x = x - (x - x^2) / (1 - 2*x)

How will you justify each result you obtain?

#Problem 1: x = x^2

[ ]:
```
import pandas as pd

initials = list([2, 0.8, 0.2])
steps = list()
approximations = list()
epsilon = 0.000000001

for i in range(3):
  x0 = initials[i]
  M = 15
  try:
    for k in range(M):
      steps.append(k)
      approximations.append(x0)
      x1 = x0**2
      if abs(x0-x1) < epsilon:
        print(f"\nWhen x0={initials[i]}, |g'({initials[i]})| < 1, so iteration␣
↪converges with tolerance of {epsilon} in {k} steps as follows:")
        break
      x0 = x1


      # print("k = ", "\t", "x= ")
    d = {'step k = ': steps, 'approximation x = ': approximations}
```

```
    df = pd.DataFrame(data=d)
    print(df)
    steps = []
    approximations = []

  except:
    print(f"\nWhen x0={initials[i]}, |g'({initials[i]})|>=1, so iteration␣
↪diverges in {k} steps as follows:")
    steps.pop()
    approximations.pop()
    d = {'step k = ': steps, 'approximation x = ': approximations}
    df = pd.DataFrame(data=d)
    print(df)
    steps = []
    approximations = []
```

```
When x0=2, |g'(2)|>=1, so iteration diverges in 14 steps as follows:
    step k =                                  approximation x =
0           0                                                 2
1           1                                                 4
2           2                                                16
3           3                                               256
4           4                                             65536
5           5                                        4294967296
6           6                                  18446744073709551616
7           7              340282366920938463463374607431768211456
8           8  115792089237316195423570985008687907853269984 6…
9           9  134078079299425970995740249982058461274793658 2…
10         10  179769313486231590772930519078902473361797697 8…
11         11  323170060713110073007148766886699519604441026 6…
12         12  104438888141315250669175271071662438257996424 9…
13         13  109074813561941592946298424473378286244826416 1…

When x0=0.8, |g'(0.8)| < 1, so iteration converges with tolerance of 1e-09 in 7
steps as follows:
   step k =   approximation x =
0          0        8.000000e-01
1          1        6.400000e-01
2          2        4.096000e-01
3          3        1.677722e-01
4          4        2.814750e-02
5          5        7.922816e-04
6          6        6.277102e-07
7          7        3.940201e-13

When x0=0.2, |g'(0.2)| < 1, so iteration converges with tolerance of 1e-09 in 4
```

```
steps as follows:
   step k =   approximation x =
0          0        2.000000e-01
1          1        4.000000e-02
2          2        1.600000e-03
3          3        2.560000e-06
4          4        6.553600e-12
```

#Problem 2: x = x^(1/2)

[ ]:
```python
import pandas as pd

initials = list([2, 0.8, 0.2])
steps = list()
approximations = list()
epsilon = 0.000001

for i in range(3):
  x0 = initials[i]
  M = 30
  try:
    for k in range(M):
      steps.append(k)
      approximations.append(x0)
      x1 = x0**(1/2)
      if abs(x0-x1) < epsilon:
        print(f"\nWhen x0={initials[i]}, |g'({initials[i]})| < 1, so iteration␣
↪converges with tolerance of {epsilon} in {k} steps as follows:")
        break
      x0 = x1


      # print("k = ", "\t", "x= ")
    d = {'step k = ': steps, 'approximation x = ': approximations}
    df = pd.DataFrame(data=d)
    print(df)
    steps = []
    approximations = []

  except:
    print(f"\nWhen x0={initials[i]}, |g'({initials[i]})|>=1, so iteration␣
↪diverges in {k} steps as follows:")
    steps.pop()
    approximations.pop()
    d = {'step k = ': steps, 'approximation x = ': approximations}
    df = pd.DataFrame(data=d)
    print(df)
    steps = []
```

```
    approximations = []
```

```
When x0=2, |g'(2)| < 1, so iteration converges with tolerance of 1e-06 in 19
steps as follows:
    step k =   approximation x =
0           0            2.000000
1           1            1.414214
2           2            1.189207
3           3            1.090508
4           4            1.044274
5           5            1.021897
6           6            1.010889
7           7            1.005430
8           8            1.002711
9           9            1.001355
10         10            1.000677
11         11            1.000339
12         12            1.000169
13         13            1.000085
14         14            1.000042
15         15            1.000021
16         16            1.000011
17         17            1.000005
18         18            1.000003
19         19            1.000001

When x0=0.8, |g'(0.8)| < 1, so iteration converges with tolerance of 1e-06 in 17
steps as follows:
    step k =   approximation x =
0           0            0.800000
1           1            0.894427
2           2            0.945742
3           3            0.972492
4           4            0.986150
5           5            0.993051
6           6            0.996519
7           7            0.998258
8           8            0.999129
9           9            0.999564
10         10            0.999782
11         11            0.999891
12         12            0.999946
13         13            0.999973
14         14            0.999986
15         15            0.999993
16         16            0.999997
```

```
17          17            0.999998

When x0=0.2, |g'(0.2)| < 1, so iteration converges with tolerance of 1e-06 in 20
steps as follows:
    step k =   approximation x =
0           0            0.200000
1           1            0.447214
2           2            0.668740
3           3            0.817765
4           4            0.904304
5           5            0.950949
6           6            0.975166
7           7            0.987505
8           8            0.993733
9           9            0.996862
10         10            0.998430
11         11            0.999214
12         12            0.999607
13         13            0.999804
14         14            0.999902
15         15            0.999951
16         16            0.999975
17         17            0.999988
18         18            0.999994
19         19            0.999997
20         20            0.999998
```

#Problem 3: x = x + 2*(x - x^2)

[ ]:
```
import pandas as pd


initials = list([2, 0.8, 0.2])
steps = list()
approximations = list()
epsilon = 0.000001

for i in range(3):
  x0 = initials[i]
  M = 10
  try:
    for k in range(M):
      steps.append(k)
      approximations.append(x0)
      x1 = x0 + 2*(x0 - x0**2)
      if abs(x0-x1) < epsilon:
        print(f"\nWhen x0={initials[i]}, |g'({initials[i]})| < 1, so iteration␣
↪converges with tolerance of {epsilon} in {k} steps as follows:")
```

```
        break
      x0 = x1
      if abs(x0)>10000000000:
        raise ValueError(f"iteration diverges in {k} steps as follows:")


      # print("k = ", "\t", "x= ")
    d = {'step k = ': steps, 'approximation x = ': approximations}
    df = pd.DataFrame(data=d)
    print(df)
    steps = []
    approximations = []

  except:
    print(f"\nWhen x0={initials[i]}, |g'({initials[i]})|>=1, so iteration␣
↪diverges in {k} steps as follows:")
    steps.pop()
    approximations.pop()
    d = {'step k = ': steps, 'approximation x = ': approximations}
    df = pd.DataFrame(data=d)
    print(df)
    steps = []
    approximations = []
```

```
When x0=2, |g'(2)|>=1, so iteration diverges in 4 steps as follows:
   step k =   approximation x =
0          0                   2
1          1                  -2
2          2                 -14
3          3                -434
   step k =   approximation x =
0          0            0.800000
1          1            1.120000
2          2            0.851200
3          3            1.104517
4          4            0.873635
5          5            1.094429
6          6            0.887738
7          7            1.087057
8          8            0.897786
9          9            1.081319
   step k =   approximation x =
0          0            0.200000
1          1            0.520000
2          2            1.019200
3          3            0.980063
```

```
4          4            1.019142
5          5            0.980125
6          6            1.019085
7          7            0.980186
8          8            1.019028
9          9            0.980247
```

#Problem 4: x = x - (x - x^2) / (1 - 2*x)

[ ]:
```
import pandas as pd

initials = list([2, 0.8, 0.2])
steps = list()
approximations = list()
epsilon = 0.000000001

for i in range(3):
  x0 = initials[i]
  M = 20
  try:
    for k in range(M):
      steps.append(k)
      approximations.append(x0)
      x1 = x0 - (x0 - x0**2) / (1 - 2*x0)
      if abs(x0-x1) < epsilon:
        print(f"\nWhen x0={initials[i]}, |g'({initials[i]})| < 1, so iteration␣
↪converges with tolerance of {epsilon} in {k} steps as follows:")
        break
      x0 = x1


      # print("k = ", "\t", "x= ")
    d = {'step k = ': steps, 'approximation x = ': approximations}
    df = pd.DataFrame(data=d)
    print(df)
    steps = []
    approximations = []

  except:
    print(f"\nWhen x0={initials[i]}, |g'({initials[i]})|>=1, so iteration␣
↪diverges in {k} steps as follows:")
    steps.pop()
    approximations.pop()
    d = {'step k = ': steps, 'approximation x = ': approximations}
    df = pd.DataFrame(data=d)
    print(df)
    steps = []
    approximations = []
```

```
When x0=2, |g'(2)| < 1, so iteration converges with tolerance of 1e-09 in 5
steps as follows:
   step k =   approximation x =
0          0            2.000000
1          1            1.333333
2          2            1.066667
3          3            1.003922
4          4            1.000015
5          5            1.000000

When x0=0.8, |g'(0.8)| < 1, so iteration converges with tolerance of 1e-09 in 4
steps as follows:
   step k =   approximation x =
0          0            0.800000
1          1            1.066667
2          2            1.003922
3          3            1.000015
4          4            1.000000

When x0=0.2, |g'(0.2)| < 1, so iteration converges with tolerance of 1e-09 in 4
steps as follows:
   step k =   approximation x =
0          0        2.000000e-01
1          1       -6.666667e-02
2          2       -3.921569e-03
3          3       -1.525902e-05
4          4       -2.328306e-10
```

[ ]:

```
```